Ingegneria del Software – 31/01/2025 – Soluzione

Consegnare SOLO i fogli del tema.	**Attaccare l'etichetta in questa zona.**
Non separarli. Altri fogli saranno ignorati.	Compiti senza etichetta non sono validi.
I fogli sono fronte/retro. 2 ore di tempo.	
Si possono consultare libri e appunti.	
Tutti i dispositivi elettronici sono vietati.	

Esercizio 1

Si consideri la seguente classe `DocumentMatcher` e la classe ausiliaria `User`.

```
public class DocumentMatcher {
  // Ritorna l'insieme di utenti che ha espresso almeno un interesse.
  public /*@ pure */ Set<User> getUsers() { .. }

  // Ritorna l'insieme di interessi di un utente. Ogni elemento di questo insieme e' a sua volta un
  // insieme di parole che dovranno far parte (tutte) di un documento perche' questo documento sia
  // utile per l'utente. In particolare, un documento e' di interesse per un utente se esiste almeno
  // un insieme di parole tra quelli restituiti che sono tutte parte del documento.
  // Solleva una IllegalArgumentException se l'utente non ha espresso interessi o se l'utente e' null.
  public /*@ pure */ Set<Set<String>> getInterests(User user) throws IllegalArgumentException { .. }

  // Restituisce l'insieme di utenti che sono interessati al documento doc.
  public /*@ pure */ Set<User> match(String doc) { .. }

  // Restituisce la lista degli utenti che sono interessati ad almeno un documento tra quelli
  // passati. La lista e' ordinata in maniera che appaiano per primi gli utenti interessati a un
  // numero maggiore di documenti.
  public /*@ pure */ List<User> matchAny(Set<String> docs) { .. }

  // Aggiunge un nuovo interesse per l'utente ovvero un nuovo insieme di parole che, se tutte parte
  // di un documento, lo rendono di interesse per l'utente. Se per l'utente non c'erano interessi
  // l'utente viene aggiunto all'insieme di utenti che hanno espresso almeno un interesse.
  // Richiede che i parametri non siano null e che nessuna delle stringhe nell'insieme interest
  // siano null, vuote o composte da piu' parole (ovvero non devono contenere spazi).
  public void addInterest(User user, Set<String> interest) { .. }
}

public class User { .. }
```

Ricordando che la classe `String` fornisce il metodo `contains(String)` che restituisce `true` se la stringa su cui il metodo è invocato contiene la stringa data e `false` altrimenti, si risponda alle seguenti domande:

Domanda a) Si specifichi in JML il metodo `match`

Soluzione

```
//@ requires doc != null
//@
//@ ensures \result != null &&
//@ (\forall User u; ; \result.contains(u) <==> ( getUsers().contains(u) &&
//@   (\exists Set<String> i; getInterests(u).contains(i) ;
//@      (\forall String s; i.contains(s) ; doc.contains(s)))))
public /*@ pure */ Set<User> match(String doc) { .. }
```

Domanda b) Si specifichi in JML il metodo `matchAny`

Soluzione

Definiamo un predicato che sfrutta il metodo `match` (puro) per contare, tra un insieme di documenti dato, quelli a cui un utente è interessato:

```
count(Set<String> docs, User u) == (\numof String doc; docs.contains(doc); match(doc).contains(u))
```

Sfruttando questo predicato possiamo specificare il metodo `matchAny` come segue:

```
//@ requires docs!=null && !docs.contains(null)
//@
//@ ensures \result != null && (\forall User u; ; \result.contains(u) <==> count(docs, u)>0) &&
//@ (\forall int i; i>=0 && i<\result.size()-1;
//@   count(docs, \result.get(i)) >= count(docs, \result.get(i+1)) &&
//@   (\forall int j; j>i && j<\result.size(); !\result.get(i).equals(\result.get(j))) )
public /*@ pure */ List<User> matchAny(Set<String> docs) { .. }
```

Domanda c) Si specifichi in JML il metodo `addInterest`

Soluzione

```
//@ requires user != null && interest != null && !interest.contains(null) &&
//@ !interest.contains("") && (\forall String s; interest.contains(s) ; !s.contains(" "))
//@
//@ ensures (\forall User u; ; getUsers().contains(u) <==> (\old(getUsers().contains(u)) || u.equals(user))) &&
//@ (\forall User u; \old(getUsers().contains(u)) && !u.equals(user) ;
//@    getInterests(u).equals(\old(getInterests(u)))) &&
//@ getInterests(user).contains(interest) &&
//@ \old(getUsers().contains(user)) ==> (getInterests(user).containsAll(\old(getInterests(user))) &&
//@    getInterests(user).size() == \old(getInterests(user).size()+1)) &&
//@ \old(!getUsers().contains(user)) ==> getInterests(user).size() == 1
public void addInterest(User user, Set<String> interest) { .. }
```

Domanda d) Si consideri un'implementazione della classe `DocumentMatcher` che fa uso di una lista ed una mappa per memorizzare utenti e relativi interessi.

```
public class DocumentMatcher {
  private List<User> users;
  private Map<User, Set<Set<String>>> interests;
  ...
}
```

Per tale implementazione, si fornisca l'invariante di rappresentazione.

Soluzione

L'invariante di rappresentazione deve dire che la lista `users` contiene gli stessi elementi dell'insieme delle chiavi della mappa `interests`, che non contiene elementi nulli e che la mappa `interests` non ha elementi nulli o elementi che contengano insiemi nulli o elementi che contengano insiemi che contengano stringhe nulle o vuote o contenenti spazi.

```
//@ private invariant
//@ users != null && interests != null && users.containsAll(interests.keySet()) &&
//@ users.size() == interests.size() && !users.contains(null) &&
//@ (\forall User u; interests.containsKey(u); interests.get(u) != null &&
//@   !interests.get(u).contains(null) && !interests.get(u).isEmpty() &&
//@   (\forall Set<String> i; interests.get(u).contains(i) ; !i.contains(null)) && !i.contains("") &&
//@     (\forall String s; i.contains(s); !s.contains(" ")) )
```

Domanda e) Si consideri una classe `DocumentMatcherExt` che modifica il metodo `matchAny` della classe originale in modo tale che se il parametro passato è `null` restituisce la lista vuota.

Può `DocumentMatcherExt` essere definita come estensione di `DocumentMatcher` in accordo con il principio di sostituzione? Si motivi la propria risposta.

Soluzione

La classe `DocumentMatcherExt` può essere definita come estensione di `DocumentMatcher` perchè ridefinisce un solo metodo rendendo più debole la sua precondizione, senza modificare la post condizione in tutti i casi in cui vale la pre-condizione della sopraclasse.

Esercizio 2

Si consideri la seguente classe `MyArrayList` che implementa una lista dinamica di interi. Internamente, la classe è implementata utilizzando un array che viene dinamicamente riallocato quando la sua capacità termina.

```
public class MyArrayList {
  private int array[];
  private int size;

  public MyArrayList(int capacity) {
    array = new int[capacity]; size = 0;
  }

  public int size() {
    synchronized(array) { return size; }
  }

  public int get(int i) {
    synchronized(array) { return array[i]; }
  }

  public void add(int num) {
    synchronized(array) {
      if (size == array.length) {
        int temp[] = new int[size * 2];
        for (int i=0; i<size; i++) { temp[i] = array[i]; }
        array = temp;
      }
      array[size] = num;
      size++;
    }
  }

  public int removeLast() throws Exception {
    synchronized(array) {
      if (size>0) {
        size--;
        return array[size];
      } else {
        throw new Exception("Empty");
      }
    }
  }
}
```

Domanda a) La classe è correttamente sincronizzata? Motivare la propria risposta e in caso di risposta negativa spiegare cosa andrebbe cambiato per evitare conflitti nell'accesso a memoria condivisa da parte di thread diversi.

Soluzione

La classe *non* è correttamente sincronizzata, in quanto l'accesso agli attributi privati è regolato mediante una sincronizzazione sull'attributo `array`, che può riferirsi a oggetti diversi nel tempo. In particolare, il metodo `add` modifica il riferimento `array` e altri thread potrebbero acquisire il lock sul nuovo oggetto referenziato da `array` prima che il metodo `add` termini.

Una semplice soluzione è quella di rendere `synchronized` tutti i metodi, rimuovendo la sincronizzazione su `array`.

Domanda b) Si modifichi il metodo `removeLast` in maniera da sospendere il chiamante in caso di lista vuota, in attesa che un numero venga inserito. Occorre modificare altri metodi? Come?

Soluzione

Assumendo di aver modificato la classe come suggerito nella soluzione al punto (a), il metodo `removeLast` diventa:

```
public synchronized int removeLast() throws InterruptedException {
  while (size==0) {
    wait();
  }
  size--;
  return array[size];
}
```

Inoltre bisogna modificare il metodo `add` inserendo l'istruzione `notifyAll()` in fondo al metodo.

Domanda c) Si considerino implementate le modifiche al punto (b) e si consideri una variabile `list` di tipo `MyArrayList`. Si scriva un frammento di codice che lancia un thread separato il quale, a sua volta, aggiunge l'elemento 0 a `list` solo se l'aggiunta avviene in una posizione pari. Se l'aggiunta è in posizione dispari, il thread si sospende in attesa dell'inserimento di nuovi elementi.

Soluzione

```
new Thread(() -> {
  synchronized(list) {
    while (list.size() % 2 == 0) {
      try { list.wait(); } catch (InterruptedException e) { }
    }
    list.add(0);
  }
}).start();
```

Esercizio 3

Si considerino le seguenti dichiarazioni di classi Java:

```
public class Hotel {
    public Set<Room> getRooms() { .. }
}

class Room {
  public int getNumPeople() { .. }

  public boolean isAvailable(Date date) { .. }

  public int getPrice() { .. }
}
```

Dove il metodo `getRooms()` ritorna l'insieme delle stanze di un `Hotel`, il metodo `getNumPeople()` ritorna il numero massimo di persone che una stanza può ospitare, il metodo `isAvailable()` ritorna la disponibilità di una stanza in una certa data e il metodo `getPrice()` il prezzo di una stanza.

Si completino i seguenti metodi *utilizzando esclusivamente i costrutti funzionali di Java*.

Domanda a) Si completi il seguente metodo, che ritorna la lista delle stanze tra gli hotel passati come parametro che sono disponibili nella data indicata e che possono ospitare il numero di persone passate per parametro. La lista ritornata deve essere ordinata in ordine di prezzo crescente.

```
public static List<Room> getAvailableRooms(Set<Hotel> hotels, Date date, int numPeople) {
  return ...
}
```

Soluzione

```
  public static List<Room> getAvailableRooms(Set<Hotel> hotels, Date date, int numPeople) {
    return hotels.stream()
      .flatMap(h -> h.getRooms().stream())
      .filter(r -> r.isAvailable(date))
      .filter(r -> r.getNumPeople() <= numPeople)
      .sorted(Comparator.comparingInt(Room::getPrice))
      .toList();
  }
```

Domanda b) Si completi il seguente metodo, che ritorna la lista degli hotel che hanno almeno una stanza che può ospitare il numero di persone passate per parametro e che è disponibile nella data passata per parametro.

```
  public static List<Hotel> getAvailableHotels(Set<Hotel> hotels, Date date, int numPeople) {
    return ...
  }
```

Soluzione

```
public static List<Hotel> getAvailableHotels(Set<Hotel> hotels, Date date, int numPeople) {
  return hotels.stream()
    .filter(h -> h.getRooms().stream()
    .anyMatch(r -> r.getNumPeople() >= numPeople && r.isAvailable(date)))
    .toList();
}
```

Soluzione alternativa senza `anyMatch` e con la `collect`.

```
public static List<Hotel> getAvailableHotels(Set<Hotel> hotels, Date date, int numPeople) {
    return hotels.stream()
    .filter(h -> h.getRooms().stream()
        .filter(room -> room.getNumPeople() >= numPeople) // Filtra per numero di persone
        .filter(room -> room.isAvailable(date))
        .count() > 0) // Se c'e' almeno una stanza valida, l'hotel e' valido
    .collect(Collectors.toList());
}
```

Esercizio 4

Si consideri il seguente metodo:

```
1  public static int testMe(int a, int b) {
2    if (a > 0 && b < 2) {
3      for (int i=0; i < b; i++) {
4        a++;
5      }
6    } else if (a > 0 && b < 3) {
7      for (int i=0; i < b; i++) {
8        b--;
9      }
10   } else {
11     a++;
12     b++;
13   }
14   return a + b;
15 }
```

Domanda a) Si determini un insieme *minimo* di test per `a` e `b` per garantire la copertura delle istruzioni (statement coverage).

Soluzione

Visto che l'`if` esterno ha tre rami, sono necessari almeno tre casi di test. Tre casi sono anche sufficienti per entrare nei due cicli `for` ed eseguire le istruzioni contenute.

(`a`=1, `b`=1) copre il primo ramo, (`a`=1, `b`=2) copre il secondo ramo, (`a`=0, `b` qualsiasi) copre il terzo ramo.

Domanda b) Si determini un insieme *minimo* di test per `a` e `b` per garantire la copertura delle decisioni e delle condizioni (edge and condition coverage).

Soluzione

I casi precedenti coprono tutte le condizioni del primo `if`, ma non coprono la condizione `a>0` vera e `b<3` falsa del secondo `if`. Per farlo, occorre aggiungere un quarto caso di test: (`a`=1, `b`=3)

Domanda c) Si determini un insieme *minimo* di test per `a` e `b` per garantire la copertura dei cammini (path coverage).

Soluzione

Quando si entra nel primo `if`, il ciclo `for` può essere eseguito una volta (caso di test (`a`=1, `b`=1)) o nessuna (caso di test (`a`=1, `b`=0)).

Quando si entra nel ramo `else if`, il ciclo `for` viene necessariamente eseguito una sola volta (caso di test (`a`=1, `b`=2)).

Un ultimo caso di test permette di eseguire il ramo `else`: ad esempio (`a`=0, `b`=qualsiasi).

Sono quindi necessari e sufficienti 4 casi di test.